

第六讲：服务层组件连接器的设计与实现

知春路遇上八里桥

<2024-05-23 Thu>



1 前情提要

2 数据结构

3 代码分析



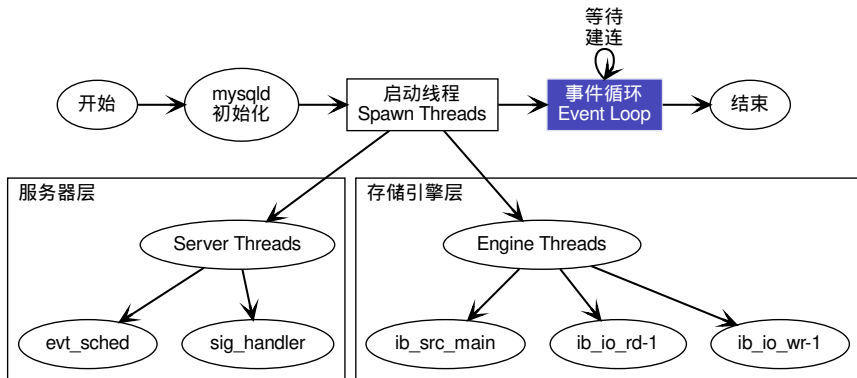
1

前情提要



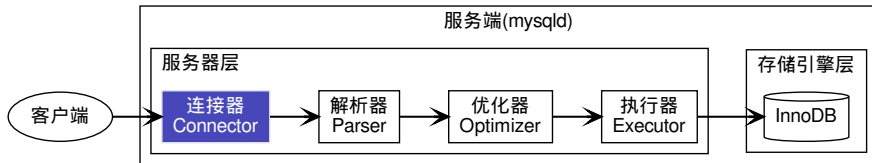
启动流程

MySQL 初始化完成之后，便进入一个死循环中，接受客户端请求，并完成客户端的命令



执行流程

- 客户端发送 QUERY 字符串后，首先就和连接器进行交互，所以我们先介绍连接器的实现¹



¹查询缓存在 8.0.x 已经移除，这里简历了流程图

2

数据结构



再看事件循环

- 调用连接器中实现的事件循环函数

- ▶ `sql/conn_handler/connection_acceptor.h`

```
61 void connection_event_loop() {
62     Connection_handler_manager *mgr =
63         Connection_handler_manager::get_instance();
64     while (!connection_events_loop_aborted()) {
65         Channel_info *channel_info = m_listener->listen_for_connection_event();
66         if (channel_info != nullptr) mgr->process_new_connection(channel_info);
67     }
68 }
```

- ▶ 需要关注以下两点 ↓

- ① `m_listener->listen_for_connection_event()` 监听并接受连接，这里返回 `Channel_info` 对象
- ② `process_new_connection()` 处理一个新来的连接

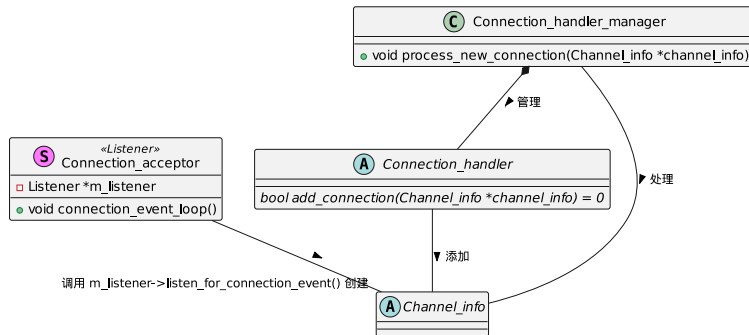
- 连接器的实现基本在目录 `sql/conn_handler` 中

- ▶ `channel_info.h`
- ▶ `connection_acceptor.h`
- ▶ `socket_connection.h`
- ▶ ...



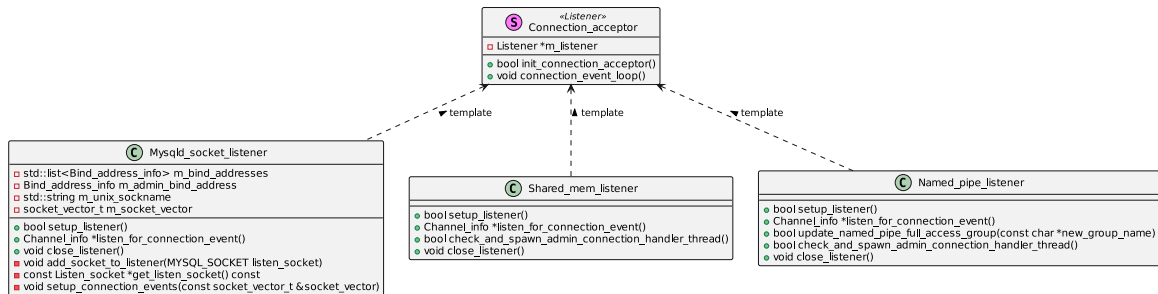
基本处理步骤及核心类

- ① Connection_acceptor 用于监听客户端的连接请求
- ② Connection_acceptor 创建 Channel_Info 作为连接信道
- ③ Connection_handler_manager 指派给 Connection_Handler 进行调度
- ④ 这吐槽一下 MySQL 的代码风格²



²https://dev.mysql.com/doc/dev/mysql-server/8.0.37/PAGE_CODING_GUIDELINES.html

Listener 监听器



- **Connection_acceptor** 以模版方式实现了 Listener 监听器,

► `sql/conn_handler/connection_acceptor.h`

```
42  template <typename Listener>
43  class Connection_acceptor {
44      Listener *m_listener;
```

- **Mysql_socket_listener** 实现 Socket 的方式监听客户端的连接事件
- **Shared_mem_listener** 通过共享内存的监听方式
- **Named_pipe_listener** 通过命名管道来监听和接收客户请求



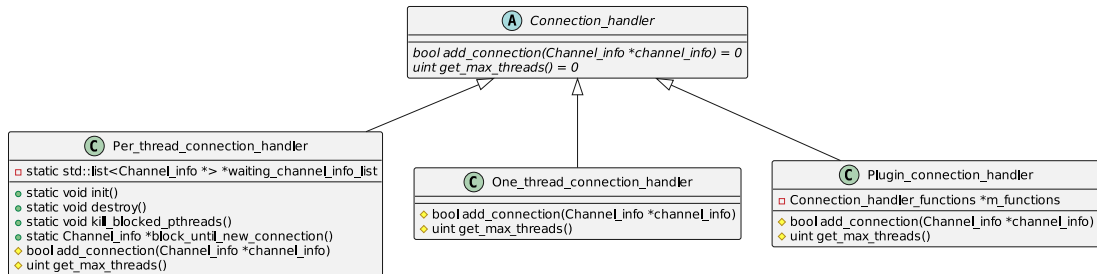
Connection_handler_manager 连接处理器的管理器

Ⓒ Connection_handler_manager
□ static Connection_handler_manager* m_instance □ Connection_handler *m_connection_handler □ Connection_handler *m_saved_connection_handler
● static Connection_handler_manager* get_instance() ● static bool init() ● void process_new_connection(Channel_info *channel_info) ● void load_connection_handler(Connection_handler *conn_handler) ● bool unload_connection_handler()

- Connection_handler_manager 是单例类³，用来管理连接处理器
 - ▶ 静态变量 m_instance 保存该单例的实例
 - ▶ 通过静态方法 get_instance() 可以获取唯一实例
- init() 初始化管理器
- process_new_connection() 处理新的请求
- load_connection_handler(Connection_handler *conn_handler) 装载处理器
- unload_connection_handler() 卸载处理器

³https://dev.mysql.com/doc/dev/mysql-server/8.0.37/classConnection__handler__manager.html

Connection_handler 连接处理器

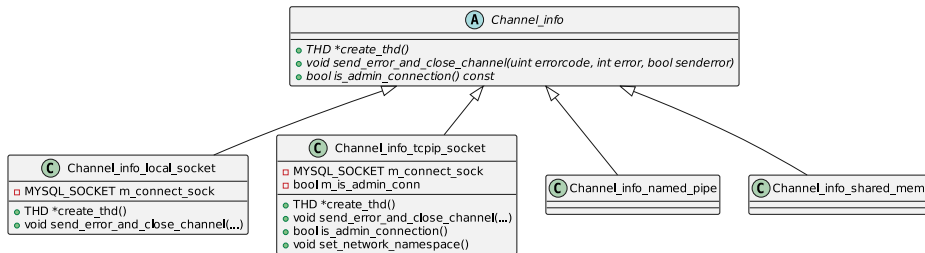


- Channel_handler 连接处理器的抽象类⁴
- 通过 thread_handling 参数可以设置线程池的类型
 - ▶ one-thread-per-connection 不启用线程池【默认】
 - ▶ pool-of-threads 启用线程池
- Per_thread_connection_handler 不启用线程池，每一个连接用单独的线程处理
- One_thread_connection_handler 启用线程池，所有连接用同一个线程处理
- Plugin_connection_handler 支持由 Plugin 具体实现的 handler，例如线程池

⁴https://dev.mysql.com/doc/dev/mysql-server/8.0.37/classConnection__handler.html



Channel_info 连接信道



- Channel_info 连接信道的抽象类⁵，包含以下四个具体实现类
- Channel_info_local_socket 与本地方式与服务器进行交互
- Channel_info_tcpip_socket 以 TCP/IP 方式与服务器进行交互
 - ▶ TCP_socket 描述 TCP socket 信息
 - ▶ Unix_socket 描述 Unix socket 信息
- Channel_info_named_pipe 命名管道的信道
- Channel_info_shared_mem 共享内存模式的信道

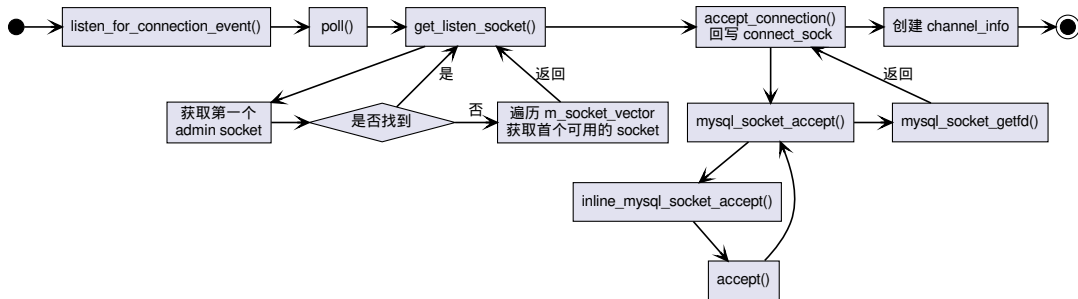
⁵https://dev.mysql.com/doc/dev/mysql-server/8.0.37/classChannel___info.html

3

代码分析



监听器处理流程梳理



监听器实现 Mysqlsocket_listener

- 获取请求，通过 poll 方式来实现多路复用获取 socket

▶ `sql/conn_handler/socket_connection.cc`

```
1348 Channel_info *Mysqlsocket_listener::listen_for_connection_event() {
1349     #ifdef HAVE_POLL
1350         int retval = poll(&m_poll_info.m_fds[0], m_socket_vector.size(), -1);
1351     #else
1352         m_select_info.m_read_fds = m_select_info.m_client_fds;
1353         int retval = select((int)m_select_info.m_max_used_connection,
1354                             &m_select_info.m_read_fds, 0, 0, 0);
1355     #endif
```

- 接收请求，获取有效的 socket 后进入 accept_connection()

▶ `sql/conn_handler/socket_connection.cc`

```
1398     if (accept_connection(listen_socket->m_socket, &connect_sock)) {
1399     #ifdef HAVE_SETNS
1400         if (!network_namespace_for_listening_socket.empty())
1401             (void)restore_original_network_namespace();
1402     #endif
1403         return nullptr;
1404     }
```

- 在 connection_event_loop() 死循环提供监听逻辑



管理器初始化

- `init()` 初始化管理器，在服务器启动时调用

- ▶ `sql/mysql.cc`

```
11202  if (Connection_handler_manager::init()) {
11203      LogErr(ERROR_LEVEL, ER_CONNECTION_HANDLING_OOM);
11204      return 1;
11205  }
```

- 具体调用栈见下图

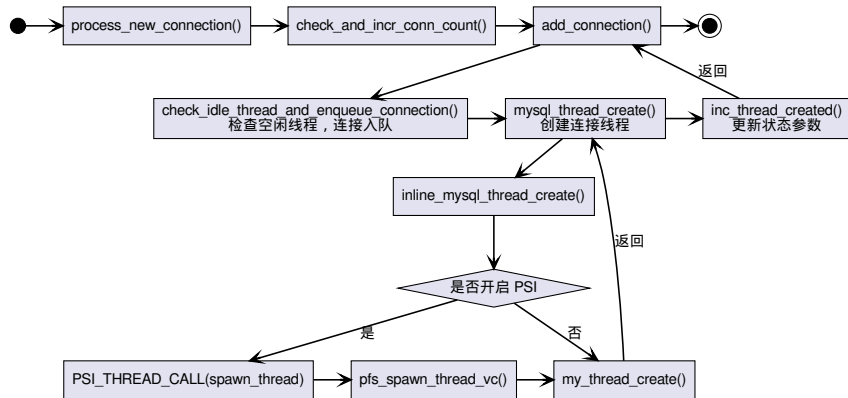
```
(gdb) i b
Num      Type      Disp Enb Address      What
1        breakpoint keep y 0x00005555913804b in Connection_handler_manager::init() at /opt/src/mysql-server/sql/con
breakpoint already hit 1 time

(gdb) bt
#0 Connection_handler_manager::init () at /opt/src/mysql-server/sql/conn_handler/connection_handler_manager.cc:150
#1 0x0000555558ba0eac in get_options (argc_ptr=0x55555e630470 <remaining_argc>, argv_ptr=0x55555e630478 <remaining_argv>)
#2 0x0000555558b8e39f in init_common_variables () at /opt/src/mysql-server/sql/mysql.cc:4930
#3 0x0000555558b989de in mysqld_main (argc=9, argv=0x55555eca0350) at /opt/src/mysql-server/sql/mysql.cc:7682
#4 0x0000555558b8590d in main (argc=2, argv=0x7fffffff088) at /opt/src/mysql-server/sql/main.cc:26

(gdb) █
```



处理新连接流程梳理



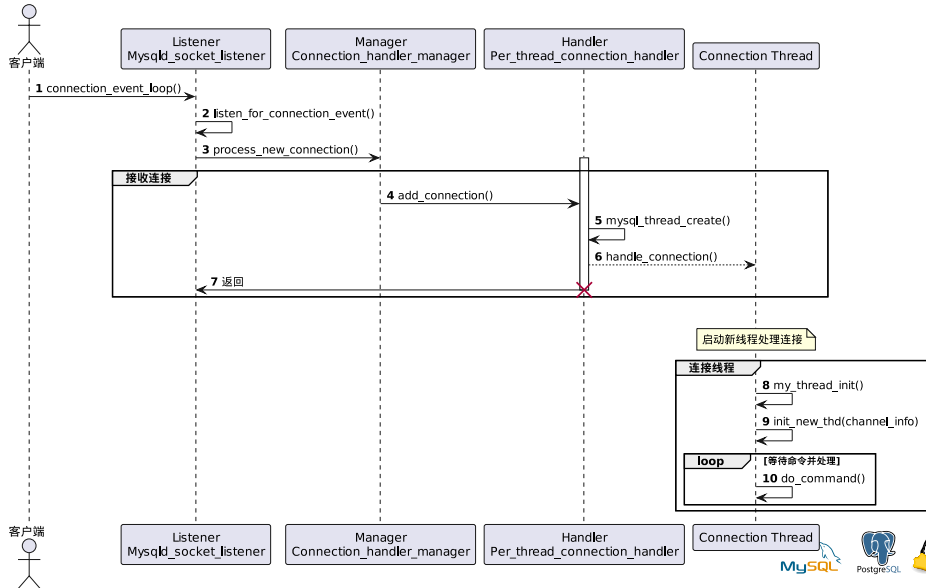
处理新连接的实现

- `check_and_incr_conn_count()` 增加连接计数，如果超过连接上限则拒绝
- `Per_thread_connection_handler::add_connection()` 添加当前信道
 - ▶ 如果有空闲的线程，那么将直接利用空闲的线程
 - ▶ 否则将创建一个新的线程来处理新连接
- `mysql_thread_create()` 创建并启动线程
 - ▶ 通过宏定义调用 `inline_mysql_thread_create()`
 - ▶ `PSI_THREAD_CALL(spawn_thread)` 宏调用 `pfs_spawn_thread_vc()`
 - ① 添加一些仪表盘的设置
 - ▶ 调用 `my_thread_create()` 后通过 `pthread_create()` 创建新线程
- `handle_connection()` 作为线程的入口函数
 - ▶ 首先是初始化线程需要的内存
 - ▶ 然后创建一个 THD 对象 `init_new_thd()`
 - ▶ 创建/或重用 `psi` 对象，并加到 `thd` 对象
 - ▶ 将 `thd` 对象加到 `thd manager` 中
 - ▶ 调用 `thd_prepare_connection()` 做验证
- 最后调用 `do_command()` 处理命令
 - ▶ `sql/conn_handler/connection_handler_per_thread.cc.h`

```
302 while (thd_connection_alive(thd)) {  
303     if (do_command(thd)) break;  
304 }
```



连接建立极简流程



结束

